Archivara Agent: Tail-Panel Regularity and a Cellar-Automata No-Go for Hadamard Order 668

Archivara research pipeline via Codex CLI

March 18, 2026

Problem. Evaluate whether a literal "cellar automata" interpretation can materially advance the search for a Hadamard matrix of order 668 once the repository's earlier flat cellular-automata branches have already failed under matched controls.

Validated scope. The executed repository contains three evidence-bearing stages: the exact-167/80 support-space branch H1, the structured mod-64 seed-repair branch H2, and a Phase 6 literal cellar reopen implemented as a canonical tail-panel automaton with a matched static boundary-debt comparator. The surviving contribution of this paper is the exact no-go result for that literal cellar encoding.

Manuscript type. Reproducible negative-result paper with exact artifact provenance, complete proofs for the implemented tail-panel encoding, and programmatically generated figures backed by checked-in JSON artifacts and deterministic control-panel recomputation.

Abstract

We study a repository-level attempt to solve the Hadamard order-668 problem using the phrase "cellar automata" as a design prompt, subject to a strict evidence rule: every substantive claim must be justified either by a formal proof or by a reproducible measurement on checked-in artifacts. The repository first interpreted the prompt as ordinary cellular automata and ran two matched-control branches on the exact length-167, weight-80 cyclic obstruction of Constantine and Constantine [2025] and the published mod-64 order-668 seed of Eliahou [2025]. Those branches do not survive review: on the exact-167/80 target, the matched non-CA baseline `direct_greedy` beats `parallel_gain_ca` on all 24/24 paired starts, and on the decisive real mod-64 seed repair test the two methods tie exactly on the best achieved modulus, ℓ_1 defect, defect count, and maximum defect magnitude.

We then execute the literal cellar interpretation instead of leaving it as a slogan. The implemented Phase 6 branch fixes a canonical tail-panel encoding for binary supports, scans the free suffix left-to-right, and maintains a dyadic pushdown-style stack of block summaries together with a matched non-stack residual state, the *boundary-debt summary*. On this encoding we prove an exact decomposition theorem for the periodic half-autocorrelation, show that every prefix coordinate is exposed to some unread-bit interaction at every cut, and prove that the boundary-debt state determines the exact completion set of any prefix. Consequently the exact extendability problem for the implemented panels collapses to finite residual state memory, and visibly pushdown memory is unnecessary for exact correctness on this encoding.

The empirical packet is also negative. On solved same-template control panels of free-tail lengths 10, 12, and 14, the exact extendable frontier has sizes 11, 13, and 15, and the matched static comparator attains exactly the same frontier in every case with zero false negatives. Moreover the boundary-debt signature is already prefix-identifying on those panels, with group counts equal to all weight-feasible prefixes: 494, 2001, and 4367. On four exact-167/80 tail panels selected from the best recorded H1 states and on four degraded order-668 seed projections, the analysis scans between 1000 and 3431 weight-feasible prefixes per panel and finds zero exact completions, zero boundary-debt frontier, and zero cellar-stack frontier. The strongest justified conclusion is therefore narrow but clean: under the exact tail-panel encoding implemented here, literal cellar memory yields no surviving advantage over the matched static residual state, so the branch should be retired rather than promoted.

1 Introduction

Recent work gives the unresolved Hadamard-668 search problem two unusually sharp exact anchors [Eliahou, 2025]. On the cyclic side, Constantine and Constantine [2025] isolate an unresolved length-167, weight-80 support with a fully specified periodic autocorrelation target. On the modular side, Eliahou [2025] construct a 64-modular Hadamard matrix of order 668, providing a concrete structured near-solution but not an exact Hadamard certificate. Those anchors are ideal for controlled method evaluation because they freeze both the state representation and the verifier before search begins.

The repository studied here used those anchors in two stages. The first stage implemented two flat cellular-automata style local-search branches. The first branch, H1, worked directly in the support space of the exact 167/80 cyclic obstruction. The second branch, H2, worked in the structured (q, s) representation induced by the published mod-64 seed. Both branches were paired with a matched same-representation non-CA baseline, `direct_greedy`, and both failed under the repository's pre-registered kill rules. That left one materially different reopen candidate: interpret "cellar automata" literally as a pushdown-style prefix automaton over exact tail panels instead of as

another local-update schedule on the same neighborhoods.[1]

This paper is about that literal reopen. The repository's Phase 6 work did not merely rename the branch. It fixed a canonical dihedral representative of each support, exposed only a contiguous free suffix, scanned that suffix left-to-right, and maintained two matched residual representations: a dyadic cellar stack and a static boundary-debt summary.[2] The key scientific question is not whether the stack is evocative, but whether it carries exact completion information that the matched static summary does not. The paper answers that question both formally and empirically for the encoding that was actually implemented.

Our contributions are:

1. We formalize the executed literal cellar branch as an exact tail-panel automaton with canonical tokenization, dyadic stack updates, a matched static boundary-debt comparator, and a shared exact tail-completion oracle.

2. We prove complete decomposition and sufficiency results for the implemented encoding: every assigned coordinate is exposed at every cut, the periodic half-autocorrelation splits into assigned, crossing, wrap, and suffix-internal terms, and the boundary-debt state determines the exact completion set of any prefix.

3. We preserve the earlier H1/H2 packet as context and show why the literal cellar reopen was warranted at all: the support-space CA loses decisively to its matched baseline, while the structured seed-repair CA ties on the only real order-668 seed test.

4. We report an exact no-go for the implemented cellar encoding. On solved 4×79 control panels of free-tail lengths 10, 12, and 14, the static frontier is already exact and prefix-identifying; on four exact-167/80 tail panels and four degraded order-668 seed projections, no exact completion or stack-only frontier survives.

5. We isolate what remains open. The paper does *not* prove that all weaker static summaries are equivalent, that pushdown memory can never help under another tokenization, or that transfer- or proof-reuse-based reopenings are impossible. It proves only that the specific exact tail-panel encoding implemented in this repository has no surviving cellar advantage.

The paper is organized as follows. Section 2 positions the work against exact Hadamard search, CA-based design generation, automata theory, and direct Hadamard heuristics. Section 3 defines the exact cyclic, modular, and tail-panel objects. Section 4 gives the executed algorithms, control panels, and complexity analysis. Section 5 proves the exact residual-state results for the implemented cellar encoding. Sections 6 and 7 describe the experiments and their outcomes. Section 8 states the limitations, open questions, and implications.

[1] Repository evidence: `results/research_context.json`; `results/concept_evolve/recurrent_state.json`; `results/concept_evolve/bridge_candidates.json`.

[2] Repository evidence: `results/swarm/director_brief.md`; `results/swarm/hypotheses.json`; `results/analysis/cellar_no_go.md`.

2 Related Work

2.1 Exact Hadamard anchors and structural search

The paper does not propose a new Hadamard construction. Its exact objects are imported from the primary order-668 anchors. The unresolved length-167, weight-80 cyclic target comes from Constantine and Constantine [2025]; the structured mod-64 seed comes from Eliahou [2025]. We use them only as fixed problem instances and verifiers, not as claims of new reduction or new modular construction.

The novelty boundary is therefore set by exact and structured Hadamard search, not by the word "automata" in isolation. The closest structured-search comparators are the SAT+CAS enumeration line of Bright et al. [2018], Williamson-type circulant construction work such as Fitzpatrick and O'Keeffe [2023], and Goethals–Seidel difference-family search such as Djoković and Kotsireas [2018]. Methodologically, these are the lines against which a symbolic or prefix-based Hadamard search method risks collapsing into exact pruning or family-parameter search. Broader modular-Hadamard background is summarized by Eliahou and Kervaire [2005] and Horadam [2007].

2.2 Cellular automata and automata-theoretic context

Cellular automata are already established in adjacent design-generation tasks, so this paper does not claim that "automata meet combinatorial design" is new. The survey of Manzoni et al. [2025] documents that broader context. Orthogonal Latin-square construction from cellular-automaton dynamics appears in Mariot et al. [2020], and CA-guided search for bent and semi-bent Boolean functions appears in Gadouleau et al. [2020], Mariot et al. [2022]. Those papers do not solve the order-668 problem, but they are enough to block any broad novelty claim at the level of CA-based discrete search itself.

The literal Phase 6 branch is closer in spirit to automata theory than to flat local CA dynamics. The repository explicitly implements a visibly pushdown-style scan over suffix symbols, so the natural external vocabulary is that of Alur and Madhusudan [2004]. The substantive question, however, is more specific than the general theory of visibly pushdown languages: for the exact tail-panel encoding used here, does the pushdown stack carry any exact completion information beyond the matched static residual state? Our main theorems show that it does not.

2.3 Direct heuristic competitors for Hadamard search

The order-668 search problem already has a heuristic literature outside cellular automata. Simulated annealing, simulated quantum annealing, quantum annealing, and QAOA-style formulations have all been proposed for Hadamard search [Suksmono, 2016, 2018, Suksmono and Minato, 2019, 2022, Suksmono, 2025]. That prior art matters methodologically even though the repository does not reproduce those algorithms here. For this paper's novelty test, it means that merely wrapping a familiar local neighborhood in new language is not a publishable advance. The present paper survives only because it compares executed branches against matched same-representation controls and then narrows its conclusion to a precise no-go statement.

2.4 How this paper differs

Relative to the exact Hadamard literature, this paper contributes neither a new family nor a new exact solver. Relative to the CA-design literature, it contributes neither a general automata paradigm nor a new positive application. Relative to the direct heuristic literature, it contributes neither a better optimizer nor a broader benchmark. What it does contribute is narrower and, for that reason, more defensible: a formal and empirical demonstration that the specific literal cellar encoding implemented in this repository collapses to a matched static residual state on the exact tail panels that were actually executed.

3 Background and Preliminaries

3.1 Hadamard and modular-Hadamard context

Definition 3.1 (Hadamard matrix)**.** A Hadamard matrix of order N is a matrix $H \in \{-1, +1\}^{N \times N}$ such that $HH^\top = NI_N$.

Definition 3.2 (Modular Hadamard surrogate)**.** For a positive integer m, an m-modular Hadamard matrix of order N is a sign matrix whose Gram matrix is congruent to NI_N modulo m [Eliahou and Kervaire, 2005, Horadam, 2007]. In this repository the modular anchor is the published 64-modular order-668 seed of Eliahou [2025].

The natural block length is $n = 167$, so $668 = 4n$ and $n = 2h+1$ with $h = 83$. The cyclic obstruction and the tail-panel automaton both work over binary supports on $\mathbb{Z}/n\mathbb{Z}$. The structured modular seed works over sign sequences $(q, s) \in \{-1, +1\}^n \times \{-1, +1\}^n$.

3.2 Exact cyclic obstruction and precursor branches

Let $x \in \{0, 1\}^n$. For $1 \le k \le h$, define the periodic binary half-autocorrelation

$$c_x(k) = \sum_{i=0}^{n-1} x_i x_{i+k \bmod n}. \tag{3.1}$$

The exact H1 target fixes three binary vectors from Constantine and Constantine [2025] and asks for a fourth support of weight 80 whose half-autocorrelation matches a prescribed target vector $\tau = (\tau_1, \ldots, \tau_h)$. The branch objective is the ℓ_1 debt

$$D_1(x) = \sum_{k=1}^{h} |c_x(k) - \tau_k|. \tag{3.2}$$

Figure 1a plots the exact target vector.

The H2 precursor uses the structured mod-64 seed of Eliahou [2025]. The repository derives four sequences $A, B, C, D \in \{-1, +1\}^n$ from (q, s) and ranks states by exact certificate first, then by best two-adic modulus, then by smaller ℓ_1 defect and supporting defect statistics.[3] Figure 1b plots the nonzero combined defects of the published seed before degradation.

[3] Repository evidence: `results/artifacts/seed_668_mod64.json`; `results/analysis/paper_metrics.json`; `hadamard668/experiments.py`.

The earlier flat CA packet matters here only because it closes the obvious local-update channels before the literal cellar reopen begins. On H1, the support-space CA loses to the matched baseline on the solved 4×79 control and on the exact 167/80 target. On H2, the structured defect-transport CA ties the matched baseline exactly on the decisive real degraded order-668 seed. Sections 6 and 7 give the numerical details.

3.3 Tail panels, extendable prefixes, and completion sets

The literal cellar branch does not search the full support space directly. Instead it fixes a canonical representative and leaves only a contiguous suffix free.

Definition 3.3 (Tail panel)**.** Fix odd length $n = 2h + 1$, weight w, target half-autocorrelation vector $\tau \in \mathbb{Z}^h$, and a fixed binary prefix $p \in \{0,1\}^B$ with $B > h$. Let $m = n - B$. The associated tail panel is the exact completion problem on words of the form

$$z = py, \qquad y \in \{0,1\}^m, \tag{3.3}$$

subject to $\sum_i z_i = w$ and $c_z(k) = \tau_k$ for all $1 \le k \le h$.

Definition 3.4 (Extendable prefix and completion set)**.** Let $x \in \{0,1\}^r$ with $0 \le r \le m$, and let $u = m - r$. We say that x is *extendable* on panel P if there exists $y \in \{0,1\}^u$ such that pxy has weight w and satisfies the exact target. The completion set of x is

$$\mathrm{Comp}_P(x) = \{y \in \{0,1\}^u : pxy \text{ has weight } w \text{ and } c_{pxy} = \tau\}. \tag{3.4}$$

We also write $\mathrm{Ext}_P(x)$ for the predicate $\mathrm{Comp}_P(x) \neq \varnothing$.

The Phase 6 control family uses solved 4×79 panels with free-tail lengths 10, 12, and 14. The stress packet uses one exact control tail-12 panel, four tail-12 panels cut from the best recorded H1 states, and four tail-12 panels obtained by projecting degraded order-668 seeds back into the exact 167/80 support channel.[4]

3.4 Boundary-debt state and cellar stack

The executed literal cellar branch maintains two matched residual descriptions.

Definition 3.5 (Boundary-debt state)**.** Let P be a tail panel and let $x \in \{0,1\}^r$. Write $t = B + r$ for the assigned length and $u = m - r$ for the unread suffix length. Let $a = px$ be the assigned prefix and let $a0^u$ denote the full-length word obtained by padding the unread suffix with zeros. The boundary-debt state of x is

$$\mathrm{BD}_P(x) = (r, t, |a|, w - |a|, \alpha_x, \lambda, \rho_x), \tag{3.5}$$

where

$$\alpha_x(k) = c_{a0^u}(k), \qquad 1 \le k \le h, \tag{3.6}$$

$$\lambda = (p_0, p_1, \ldots, p_{h-1}), \tag{3.7}$$

$$\rho_x = (a_{t-h}, a_{t-h+1}, \ldots, a_{t-1}). \tag{3.8}$$

Thus $\mathrm{BD}_P(x)$ stores the zero-padded assigned pair vector, the fixed left boundary bits, the current right boundary bits, and the exact remaining weight.

[4] Repository evidence: `results/experiments/cellar_phase6.json`.

Algorithm 1 Exact Tail-Panel Analysis for a Fixed Panel P

Compute all exact completions $\mathrm{Comp}_P(\epsilon)$ by enumerating all tails of the required weight.
Enumerate every weight-feasible prefix x of the free suffix.
for each prefix x **do**
    Compute the exact completion index set $I_P(x)$.
    Mark x extendable iff $I_P(x) \neq \varnothing$.
    Compute the static signature $\mathrm{BD}_P(x)$.
    Compute the cellar signature $\mathrm{Cellar}_P(x)$.
end for
for each signature family $\sigma \in \{\mathrm{BD}, \mathrm{Cellar}\}$ **do**
    Group prefixes by the key $\sigma_P(x)$.
    Mark a group positive iff it contains at least one extendable prefix.
    Report frontier size as the total number of prefixes in positive groups.
    Report purity by checking whether any group mixes extendable and nonextendable prefixes or distinct completion sets.
end for

Definition 3.6 (Cellar stack state)**.** The cellar state augments $\mathrm{BD}_P(x)$ with a dyadic stack of block summaries. Reading the free suffix left-to-right, each incoming bit starts as a length-1 block; whenever the top two blocks have equal length they are merged into a parent block that stores its length, weight, bit string, boundary bit patterns, and internal linear pair vector. The resulting stack is the repository's literal "cellar" memory.

Figure 5 summarizes the executed encoding.

4 Method

4.1 From flat CA no-go to literal cellar reopen

The repository's experimental chronology matters because it explains why the literal cellar branch received budget at all. The initial H1 and H2 branches were same-space, same-budget comparisons against `direct_greedy`. The literal cellar reopen was authorized only after those branches had already failed or tied under matched controls.[5] The Phase 6 reopen therefore had a narrower burden than the earlier flat CA packet: it had to justify its *different information channel*, not merely another move-selection rule on the same neighborhood.

4.2 Exact tail-panel analysis pipeline

For a fixed tail panel P, the executed analysis computes exact completions, enumerates all weight-feasible prefixes, groups prefixes by residual signature, and measures how large the resulting frontier remains when only signatures with at least one exact completion are retained. The same pipeline is applied twice, once with the static signature BD_P and once with the cellar signature Cellar_P.

[5] Repository evidence: `results/swarm/director_brief.md`; `results/swarm/hypotheses.json`; `results/analysis/cellar_no_go.md`.

Algorithm 2 Streaming Update of the Executed Cellar State

Require: Current prefix x, current stack S, incoming bit b.

1: Append b to the free-prefix word to obtain xb.
2: Push a length-1 block summary for b onto S.
3: **while** the top two stack blocks have equal length **do**
4: Pop those two blocks.
5: Merge them into their parent block and push the merged summary.
6: **end while**
7: Update the matched boundary-debt state $\mathrm{BD}_P(xb)$.
8: Return the pair $\big(\mathrm{BD}_P(xb), S\big)$.

The exact control family additionally evaluates held-out free-tail lengths 10, 12, and 14 because the failed proof-carrying feedback layer required a 25% frontier reduction at unchanged witness retention.[6] Once the matched static frontier is already exact on that control family, the feedback objective becomes impossible by construction.

4.3 Cellar update law

The dynamic part of the literal cellar branch is the streaming construction of the dyadic stack. The acceptance test itself remains exact and external: a prefix is accepted only when the matched exact completion oracle says it remains extendable.

The key fairness property is that the cellar stack never receives a stronger exact oracle than the static baseline. Both views use the same canonical tokenization, the same exact completion test, the same control panels, and the same accounting of canonical prefixes. The only difference is memory discipline.

4.4 Experimental packets

Table 1 summarizes the packets used in the final paper. The earlier H1/H2 packet supplies the precursor no-go context. The literal cellar packet consists of an exact control family and a real-anchor stress family.

4.5 Complexity analysis

Let m be the free-tail length and let R be the required number of ones in that tail. The exact completion oracle in Algorithm 1 enumerates $\binom{m}{R}$ tails and evaluates each full word by a length-n half-autocorrelation computation, so the dominant exact-oracle cost is

$$O\left(\binom{m}{R} nh\right) = O\left(\binom{m}{R} n^2\right) \tag{4.1}$$

for the naive implementation used here. The number of weight-feasible prefixes is at most 2^m and equals 494, 2001, and 4367 on the executed control family with $m \in \{10, 12, 14\}$. Computing $\mathrm{BD}_P(x)$

[6] Repository evidence: `research_rubric.json`; `results/analysis/cellar_no_go.md`.

Table 1: Exact packets used in the revised manuscript. The cellar-specific contribution is restricted to the last two rows.

Packet	Exact anchor	Representation	Panel count	Endpoint
H1 control sweep	Solved 4×79 control from Constantine and Constantine [2025]	Weight-43 support search	16 starts	Exact witness or best ℓ_1 debt
H1 target sweep	Exact 167/80 obstruction from Constantine and Constantine [2025]	Weight-80 support search	24 starts	Exact witness or best ℓ_1 debt
H2 packet	Published mod-64 seed from Eliahou [2025]	Structured (q, s) repair	$3 + 1$ starts	Exact repair or modulus lift
Cellar control family	Solved 4×79 tail panels	Canonical suffix prefixes	3 panels	Witness retention and frontier exactness
Cellar real-anchor family	Best H1 tail panels + degraded 668 seed projections	Canonical suffix prefixes	8 panels	Exact completion, frontier reduction, or stack-only split

for one prefix costs $O(nh)$ because the code recomputes the zero-padded half-autocorrelation directly. Building the dyadic stack from scratch for one length-r prefix costs $O(hr \log r)$ in the implemented block-summary construction because each level of merges processes total block length $O(r)$ across $O(\log r)$ levels. Since the executed panels keep $m \leq 14$, the exhaustive oracle rather than the streaming update dominates runtime in practice.

5 Theoretical Results and Proofs

The theoretical purpose of this section is not to prove a general theorem about all possible cellar-like representations. It is to prove, completely and explicitly, what the implemented boundary-debt state does and does not forget.

5.1 Exposure and decomposition

Theorem 5.1 (Every assigned coordinate is exposed at every cut)**.** *Let $n = 2h + 1$ be odd, and let t satisfy $1 \leq t < n$. For each shift $k \in \{1, \ldots, h\}$ define the left and right exposure windows*

$$L_k = [0, k-1], \qquad R_k = [t-k, t-1], \tag{5.1}$$

where R_k is intersected with $[0, t-1]$ when $t - k < 0$. Then

$$\bigcup_{k=1}^{h} (L_k \cup R_k) = [0, t-1]. \tag{5.2}$$

Equivalently, every assigned prefix index belongs to some left or right boundary window for a half-correlation shift no larger than h.

Proof. We compute the two unions separately.

First,

$$\bigcup_{k=1}^{h} L_k = [0, h-1], \tag{5.3}$$

because the intervals are nested and the largest one is $L_h = [0, h-1]$.

Second, if $t - k < 0$ then $R_k \cap [0, t-1] = [0, t-1]$, so the claim is immediate. It therefore suffices to consider the case $t - k \geq 0$. In that case the right windows are also nested as k grows, and their union is

$$\bigcup_{k=1}^{h} R_k = [\max(0, t-h), t-1]. \tag{5.4}$$

We now distinguish two cases.

Case 1: $t \leq h$**.** Then $[0, h-1] \supseteq [0, t-1]$, so the left-window union alone already covers the full assigned prefix.

Case 2: $h < t < 2h+1$**.** Since $t \leq 2h$, we have $t - h \leq h$. Therefore the two covering intervals

$$[0, h-1] \quad \text{and} \quad [t-h, t-1] \tag{5.5}$$

overlap or touch: the left interval ends at $h-1$ and the right interval begins at $t - h \leq h$. Hence their union is the full interval $[0, t-1]$.

In both cases the union over all left and right boundary windows equals $[0, t-1]$. This proves that every assigned coordinate is exposed by at least one half-correlation shift. □

Lemma 5.2 (Exact decomposition of the half-autocorrelation at a tail cut)**.** *Fix a tail panel P of odd length $n = 2h+1$ with fixed prefix $p \in \{0,1\}^B$, free-tail length $m = n - B$, and target vector τ. Let $x \in \{0,1\}^r$ be a free-prefix assignment, let $u = m - r$, and let $y \in \{0,1\}^u$ be a completion candidate. Write $a = px$ for the assigned prefix of length $t = B + r$, and define*

$$\alpha_x(k) = c_{a0^u}(k), \qquad 1 \leq k \leq h. \tag{5.6}$$

For each shift $k \in \{1, \ldots, h\}$,

$$c_{axy}(k) = \alpha_x(k) + \sum_{j=0}^{k-1} a_{t-k+j} y_j \tag{5.7}$$

$$+ \sum_{j=0}^{u-k-1} y_j y_{j+k} \tag{5.8}$$

$$+ \sum_{j=0}^{k-1} y_{u-k+j} p_j. \tag{5.9}$$

When $u < k$, the middle sum is empty and contributes zero.

Proof. Fix $k \in \{1, \ldots, h\}$. By definition,

$$c_{axy}(k) = \sum_{i=0}^{n-1} z_i z_{i+k \bmod n}, \tag{5.10}$$

where $z = axy$ is the full completed word.

We partition the index set $\{0, 1, \ldots, n-1\}$ according to the location of the pair $(i, i+k \bmod n)$ relative to the cut at position t.

1. Fully assigned terms. If both endpoints lie in the assigned region $[0, t-1]$, then the corresponding contribution is already present in the zero-padded word $a0^u$. Summing all such terms yields exactly $\alpha_x(k)$.

2. Crossing terms from assigned to unread suffix. These are the indices i for which $i \in [0, t-1]$ and $i+k \in [t, n-1]$. The second condition is equivalent to $i \in [t-k, t-1]$. Because $k \le h < B \le t$, the interval $[t-k, t-1]$ lies entirely inside the assigned region. Writing $i = t-k+j$ with $0 \le j \le k-1$, the contribution becomes

$$z_i z_{i+k} = a_{t-k+j} y_j. \tag{5.11}$$

Summing these k terms gives Eq. (5.7).

3. Suffix-internal terms. These are the indices i for which both i and $i+k$ lie in the unread suffix interval $[t, n-1]$. Writing $i = t+j$, this requires $0 \le j \le u-k-1$. The contribution is then $y_j y_{j+k}$, and summing over all such j gives Eq. (5.8). If $u < k$ there are no such indices and the sum is empty.

4. Wrap terms from unread suffix back into the fixed left boundary. These are the indices i in the unread suffix for which $i+k \ge n$, so the second endpoint wraps to the beginning of the word. Such indices are exactly $i = n-k, \ldots, n-1$. Because $n = t+u$, we can write them as $i = t+u-k+j$ with $0 \le j \le k-1$. The wrapped endpoint is then

$$i + k - n = j, \tag{5.12}$$

which lies in the fixed left boundary because $j < k \le h < B$. Therefore the corresponding contribution is

$$z_i z_{i+k \bmod n} = y_{u-k+j} p_j, \tag{5.13}$$

and summing over $j = 0, \ldots, k-1$ yields Eq. (5.9).

These four cases are exhaustive, because every cyclic pair either stays fully in the assigned region, crosses the cut from assigned to unread, stays inside the unread suffix, or wraps from the unread suffix back to the beginning. Their contributions are disjoint, so adding them gives the stated identity. □

5.2 Exact residual-state sufficiency

Theorem 5.3 (Boundary-debt state determines the exact completion set)**.** *Let P be a fixed tail panel, and let x and x' be two free prefixes on that panel. If*

$$\mathrm{BD}_P(x) = \mathrm{BD}_P(x'), \tag{5.14}$$

then the two prefixes have the same remaining length, the same remaining weight, and the same exact completion set:

$$\mathrm{Comp}_P(x) = \mathrm{Comp}_P(x'). \tag{5.15}$$

In particular,

$$\mathrm{Ext}_P(x) = \mathrm{Ext}_P(x'). \tag{5.16}$$

Proof. Let x have length r and x' have length r'. Equality of boundary-debt states includes equality of the first two coordinates r and $t = B+r$, so $r = r'$ and the common unread-tail length is $u = m-r$. Equality of the remaining-weight coordinate also guarantees that the admissible completion tails for x and x' are drawn from the same set of length-u binary words of the required weight.

Fix any such candidate completion $y \in \{0,1\}^u$. We will show that the full half-autocorrelation vectors of pxy and $px'y$ coincide componentwise. Let $k \in \{1,\dots,h\}$. By Lemma 5.2,

$$c_{pxy}(k) = \alpha_x(k) + \sum_{j=0}^{k-1} a_{t-k+j} y_j + \sum_{j=0}^{u-k-1} y_j y_{j+k} + \sum_{j=0}^{k-1} y_{u-k+j} p_j, \tag{5.17}$$

$$c_{px'y}(k) = \alpha_{x'}(k) + \sum_{j=0}^{k-1} a'_{t-k+j} y_j + \sum_{j=0}^{u-k-1} y_j y_{j+k} + \sum_{j=0}^{k-1} y_{u-k+j} p_j. \tag{5.18}$$

Because $\mathrm{BD}_P(x) = \mathrm{BD}_P(x')$, the assigned pair vectors satisfy $\alpha_x = \alpha_{x'}$. Equality of boundary-debt states also includes equality of the right boundary words ρ_x and $\rho_{x'}$, and the indices $t-k,\dots,t-1$ used in the crossing term are exactly the last k entries of those right boundary words. Hence

$$a_{t-k+j} = a'_{t-k+j} \qquad \text{for } 0 \le j \le k-1. \tag{5.19}$$

The suffix-internal term and the wrap term depend only on the common suffix y and the fixed left boundary $p_0,\dots,p_{h-1}$, which is the same for both prefixes because the panel itself is fixed. Therefore every summand on the right-hand side agrees, and so

$$c_{pxy}(k) = c_{px'y}(k) \tag{5.20}$$

for every $k \in \{1,\dots,h\}$.

Thus the full half-autocorrelation vectors of pxy and $px'y$ are identical for every admissible completion tail y. Since the target vector τ is also fixed by the panel, pxy satisfies the exact target if and only if $px'y$ does. Therefore the set of completion tails y that solve the panel is the same for x and x', which proves

$$\mathrm{Comp}_P(x) = \mathrm{Comp}_P(x'). \tag{5.21}$$

The extendability equivalence is immediate because a set is nonempty if and only if an equal set is nonempty. □

Corollary 5.4 (No stack-only extendability split on the implemented encoding)**.** *For any fixed tail panel P, if two prefixes share the same boundary-debt state, then no refinement of those prefixes by dyadic cellar stack memory can change either their extendability predicate or their exact completion set.*

Proof. The cellar state is an augmentation of the boundary-debt state; it carries strictly more information, not less. By Theorem 5.3, once the boundary-debt state is fixed, the exact completion set is already fixed. Therefore changing only the stack component while keeping boundary debt unchanged cannot change the completion set or the extendability predicate. □

Corollary 5.5 (Finite residual-state collapse for the executed panels)**.** *For any fixed executed tail panel P, the exact prefix-completion problem can be recognized by a finite-state machine whose states are the reachable boundary-debt states. In particular, visibly pushdown memory is unnecessary for exact correctness on the implemented encoding.*

Proof. Each component of $\mathrm{BD}_P(x)$ ranges over a finite set because the panel length, target, and weight are fixed. The prefix length r ranges over $\{0, 1, \ldots, m\}$. The assigned and remaining weights are bounded by w. The vector $\alpha_x \in \mathbb{Z}^h$ is computed from a fixed-length binary word, so each coordinate lies in a finite integer interval. The left and right boundary words are binary strings of fixed length h.

Reading one more suffix bit updates these quantities deterministically: the prefix length increases by one, the weights update by adding the new bit, the zero-padded assigned pair vector is recomputed from the longer assigned word, and the right boundary shifts by one coordinate. Therefore there is a deterministic transition function on reachable boundary-debt states. By Theorem 5.3, acceptance depends only on that state. Hence the executed panel family admits a finite-state recognizer and does not require pushdown memory for exact extendability. □

Proposition 5.6 (Set-theoretic impossibility of feedback improvement on an exact static frontier)**.** *Let $F \subseteq X$ be the exact extendable frontier of a control panel, and let $G \subseteq X$ be the frontier retained by a static filter. If $G = F$, then any additional filter $H \subseteq G$ that preserves all witnesses must satisfy $H = G$. Consequently no witness-preserving feedback layer can reduce frontier size below $|G|$.*

Proof. Because $G = F$, every element of G is extendable and every extendable element lies in G. Let $H \subseteq G$ be any additional filter that preserves all witnesses. Preserving all witnesses means that every extendable element of G must remain in H. But the extendable elements of G are exactly the elements of G itself, because $G = F$. Therefore every element of G must belong to H. Since $H \subseteq G$ and $G \subseteq H$, the two sets are equal. Hence $|H| = |G|$, so no further reduction is possible without losing witnesses. □

Remark 5.7. Theorems 5.1–5.3 are structural results about the implemented exact encoding. Proposition 5.6 then converts the control-family measurements of Section 7.3 into a formal impossibility statement for the repository's failed proof-carrying feedback objective.

6 Experimental Setup

6.1 Artifacts, commands, and environment

The manuscript rests on checked-in artifacts under `results/artifacts/`, `results/experiments/`, and `results/analysis/`. The canonical reproduction commands for the exact cellar branch are

```
python3 -m hadamard668.artifacts export
python3 -m hadamard668.cellar \
  --output results/experiments/cellar_phase6.json \
  --target-limit 4
python3 figures/generate_cellar_figures.py
```

while the earlier flat CA packet is reproduced by

```
python3 -m hadamard668.experiments run-all
```

We reran the cellar analysis and figure-generation steps during this writer pass, confirmed that the recomputed control-family counts match the checked-in narrative, and generated the two cellar-specific figure files together with `results/analysis/cellar_paper_metrics.json`.

The current environment reports Python 3.10.17, Linux 4.4.0 on x86_64, and 20 virtual CPUs with approximately 448 GiB of RAM. The earlier flat CA experiments already existed in checked-in JSON form, so we treat their recorded summaries as the primary evidence and the rerun cellar packet as the only writer-pass recomputation.

6.2 Precursor H1/H2 Packet

The precursor packet is inherited unchanged from the repository's earlier phases. The solved 4×79 control and exact 167/80 target sweeps each run 96 steps with the same phase move cap for `parallel_gain_ca` and `direct_greedy`. The structured $n = 9$ ladder runs 24 steps, and the real degraded order-668 seed repair packet runs 48 steps. Bootstrap 95% intervals are reported for H1 mean best distance, and Wilson 95% intervals are reported for the $n = 9$ ladder hit rates. The real order-668 seed repair packet has $N = 1$ and is therefore reported deterministically.

6.3 Literal cellar packet

The literal cellar packet contains two families.

Solved control family. Three exact tail panels are cut from the solved 4×79 control with free-tail lengths 10, 12, and 14. These are exhaustive same-template controls because the required tail weight is fixed and small, so every exact completion can be enumerated. Their role is twofold: verify zero false negatives for the literal cellar encoding and test whether the matched static comparator already saturates the exact frontier.

Real-anchor stress family. Eight tail-12 panels are analyzed against the actual order-668 anchors. Four come from the best recorded H1 target states retained by the repository's exact target sweep.[7]

```
target_167_tail12::direct_greedy::random_weight::30001
target_167_tail12::direct_greedy::random_weight::30004
target_167_tail12::direct_greedy::random_weight::30012
target_167_tail12::direct_greedy::random_weight::30011
```

The other four come from deterministic projections of degraded order-668 seeds into the exact 167/80 support channel.[8]

```
seed_projection_tail12::single_flip_0_mod8
seed_projection_tail12::single_flip_41_mod16
seed_projection_tail12::cluster3_0_2_mod8
seed_projection_tail12::cluster5_0_4_mod8
```

Every quantitative claim about the literal cellar branch is an exact count on one of these eleven panels (three solved controls plus eight real-anchor panels), and the labels and counts are taken

[7] Repository evidence: `results/experiments/h1_target_sweep.json`; `results/analysis/paper_metrics.json`.

[8] Repository evidence: `results/artifacts/seed_668_mod64.json`; `hadamard668/experiments.py`; `results/experiments/h2_seed_attempt.json`.

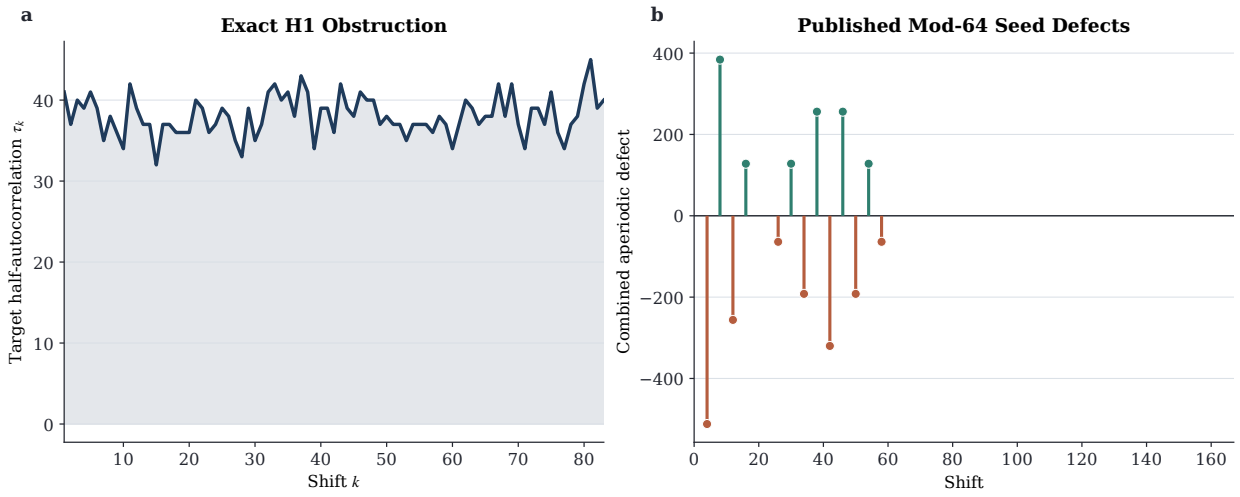

Figure 1: Exact objects that define the order-668 search problem. (a) The half-autocorrelation target vector τ for the unresolved length-167, weight-80 cyclic obstruction of Constantine and Constantine [2025]. (b) The signed nonzero combined aperiodic defects of the published mod-64 order-668 seed of Eliahou [2025]. The later literal cellar packet acts on canonical tail panels cut from the cyclic channel, but the seed-projection stress family is derived from the same modular anchor.

from the checked-in Phase 6 report plus deterministic writer-pass recomputation.[9] No confidence interval is appropriate because these are deterministic exhaustive counts rather than Monte Carlo estimates.

7 Results

7.1 Exact objects and precursor no-go context

Figure 1 shows the exact objects that define the repository's order-668 search problem. The cyclic anchor is a fixed target vector τ of length 83, not a vague similarity score. The modular anchor is a concrete signed defect spectrum for a published mod-64 seed, not a generic "promising start." Those exact anchors are why the later cellar no-go result matters: the literal reopen is evaluated against the same underlying order-668 obstruction rather than against a synthetic proxy.

The flat CA packet fails before the literal reopen begins. Table 2 and Figures 2–4 summarize the decisive outcomes. On the exact 167/80 target sweep, `direct_greedy` beats `parallel_gain_ca` on all 24/24 paired starts with a best-distance mean of 147.92 versus 188.50. On the solved 4×79 control sweep, neither serious method finds an exact witness under the locked budget, but `direct_greedy` still wins 11/16 paired starts and attains a better mean distance, 41.50 versus 48.38. On the real order-668 structured seed repair packet, `parallel_gain_ca` and `direct_greedy` tie exactly on the best achieved modulus, ℓ_1 defect, defect count, maximum defect magnitude, best step, and unique-orbit count.[10] The literal cellar branch is therefore not a coequal third positive signal; it is a

[9] Repository evidence: `results/experiments/cellar_phase6.json`; `results/analysis/cellar_paper_metrics.json`.

[10] Repository evidence: `results/experiments/h2_seed_attempt.json`; `results/analysis/paper_metrics.json`.

Table 2: Precursor packet summary. Confidence intervals are bootstrap 95% intervals for H1 and Wilson 95% intervals for the $n = 9$ ladder. The real order-668 seed packet has $N = 1$.

Packet	Comparator outcome	Exact/goal hit	Primary metric	Supporting detail
H1 control	`direct_greedy` wins 11/16 paired starts	0/16 for both	mean best debt 41.50 vs. 48.38	median orbits 13.0 vs. 4.0
H1 target	`direct_greedy` wins 24/24 paired starts	0/24 for both	mean best debt 147.92 vs. 188.50	CI $[141.83, 154.58]$ vs. $[178.17, 199.17]$
H2 ladder	weakly favors the CA rule but nondiscriminative	2/3 vs. 1/3	mean best ℓ_1 defect 2.67 vs. 10.67	random-rule control solves 3/3 starts
H2 real seed	exact tie	0/1 for both	best ℓ_1 defect 2944 for both	best modulus 16 for both

narrowed reopen after the earlier local-dynamics channels have already failed.

7.2 Theoretical predictions for the cellar branch

The formal results of Section 5 already predict what would count as a genuine cellar advantage. By Theorem 5.3, the stack can matter only if the executed boundary-debt summary fails to determine exact completion sets. By Theorem 5.1, every assigned coordinate is touched by some unread interaction at every cut, so any advantage must come from how those interactions are summarized, not from hidden unaffected coordinates. Figure 6a–b visualizes that exposure theorem for the two lengths that actually appear in the packet, 79 and 167.

7.3 Solved same-template controls: the static frontier is already exact

The decisive control-family result is not merely that the cellar stack has zero false negatives. It is that the matched static comparator is already exact on the solved control family. Table 3 and Figure 6c show the full counts for free-tail lengths 10, 12, and 14.

For tail length 10, the panel contains 494 weight-feasible prefixes and exactly one root completion. The exact extendable frontier has size 11, and both the boundary-debt and cellar frontiers also have size 11. For tail length 12, the corresponding numbers are 2001 feasible prefixes and frontier size 13. For tail length 14, they are 4367 feasible prefixes and frontier size 15.

The stronger empirical fact is that the boundary-debt signature is already prefix-identifying on all three solved controls. Its group count equals the number of feasible prefixes exactly: 494, 2001, and 4367. The cellar signature is also injective, but that is beside the point: once the static signature is already exact and injective on the solved control family, Proposition 5.6 rules out any further witness-preserving reduction by the repository's abandoned proof-carrying feedback objective. The problem is not that the static comparator is slightly too weak; it is already saturated on the validated controls.

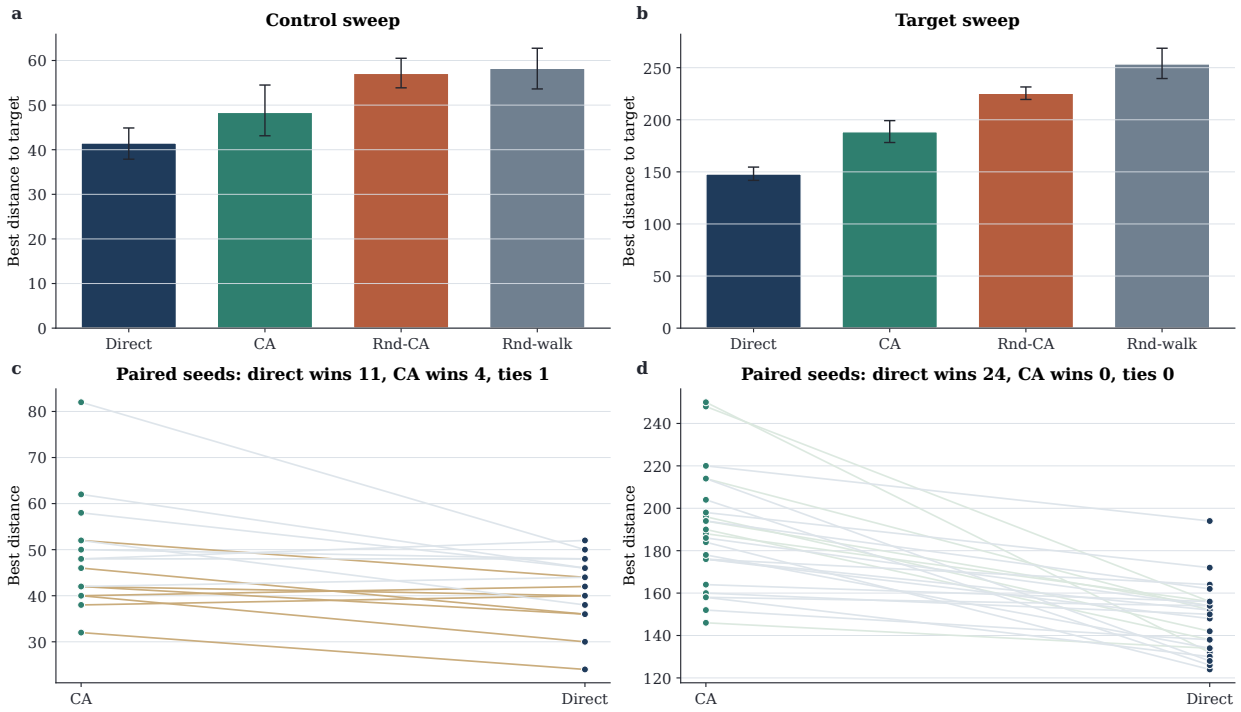

Figure 2: Precursor H1 results on the solved 4×79 control and the exact 167/80 target. The top row reports mean best distance with bootstrap 95% intervals; the bottom row gives paired seed-level comparisons between `parallel_gain_ca` and `direct_greedy`. The literal cellar reopen is motivated precisely because these same-space local-update rules do not survive this matched-control test.

Table 3: Solved same-template control family for the literal cellar branch. The boundary-debt signature is already exact and prefix-identifying on all three panels.

Panel	Feasible prefixes	Exact root completions	Exact extendable frontier	Boundary frontier	Cellar frontier	Boundary groups
4×79 tail 10	494	1	11	11	11	494
4×79 tail 12	2001	1	13	13	13	2001
4×79 tail 14	4367	1	15	15	15	4367

7.4 Real-anchor stress packet: no exact completion and no stack-only frontier

The real-anchor packet is negative in a different way. On all eight tail-12 panels selected from actual order-668 anchors, the analysis scans a nontrivial number of feasible prefixes but finds no exact completion at all. The four exact-167/80 target panels contain 1000, 2001, 3002, and 3431 feasible prefixes respectively; the four degraded seed projections contain 3431, 3431, 2001, and 2001 feasible prefixes. Yet every one of those eight panels has exact root completion count 0, boundary frontier 0, and cellar frontier 0.

Table 4 and Figure 6d make the relevant point visually: the prefix counts are not tiny, and the static signature is again injective on the recorded panels, but there is no exact frontier for either residual view to preserve. The literal cellar branch therefore achieves none of the endpoints that would justify promotion. It produces no exact witness, no new exact-prefix completion, no static-versus-stack separation, and no frontier reduction unavailable to the matched static residual state.

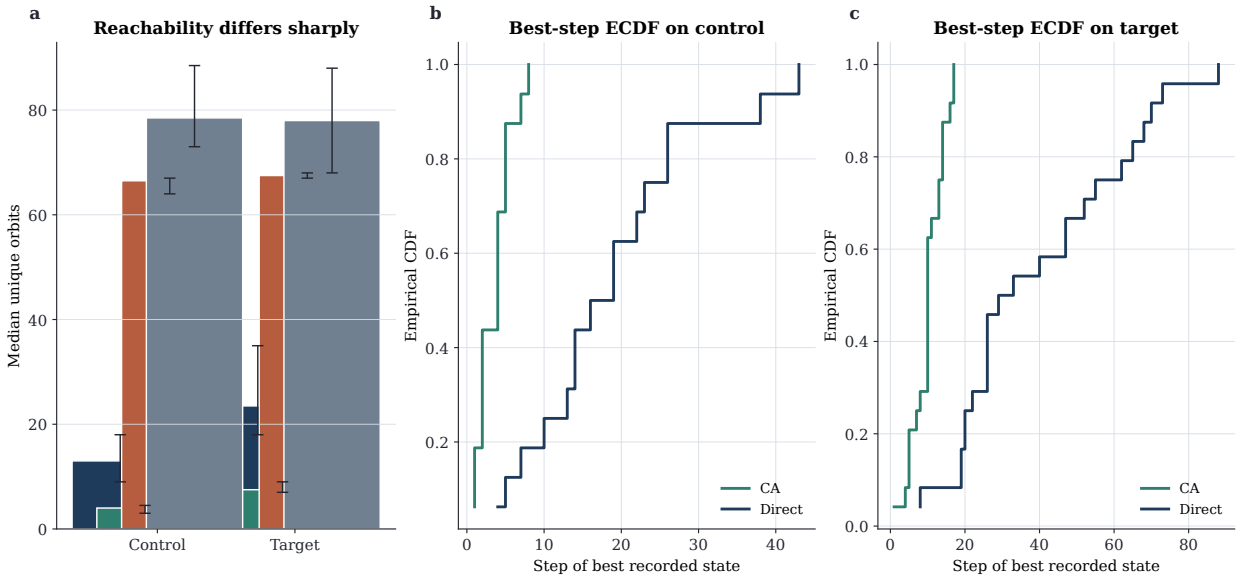

Figure 3: Reachability diagnostics for the precursor H1 packet. The CA rule visits fewer canonical orbits and reaches its best recorded state much earlier than the matched baseline, especially on the exact target. This early plateau is one of the reasons the repository stopped spending budget on flat support-space local dynamics and reopened the problem through a different information channel.

7.5 What the evidence does and does not support

The supported conclusion is precise:

> *For the exact tail-panel encoding implemented in* `hadamard668/cellar.py`*, literal cellar memory does not provide any surviving advantage over the matched static boundary-debt state on the solved control family or on the recorded order-*668 *anchor panels.*

The conclusion is strong because it combines theory and measurement. Theorems 5.3 and 5.4 show that boundary debt is already an exact residual summary for the implemented encoding. The control-family measurements show that the static frontier is not merely sound but exact on solved same-template panels. The real-anchor packet shows that the encoding also fails to surface any exact completion on the best recorded 167/80 states or on degraded order-668 seed projections.

What the evidence does *not* support is equally important. The paper does not prove that every weaker static summary is equivalent to the cellar stack. It does not prove that another tokenization, another completion oracle, or a proof-reuse channel could never help. It does not solve order 668. It retires one exact encoding cleanly.

8 Discussion

8.1 Why the no-go is informative

The literal cellar branch fails for a more informative reason than the precursor branches. In H1 and H2 the negative evidence is comparative: the CA-style local dynamics lose or tie against a matched

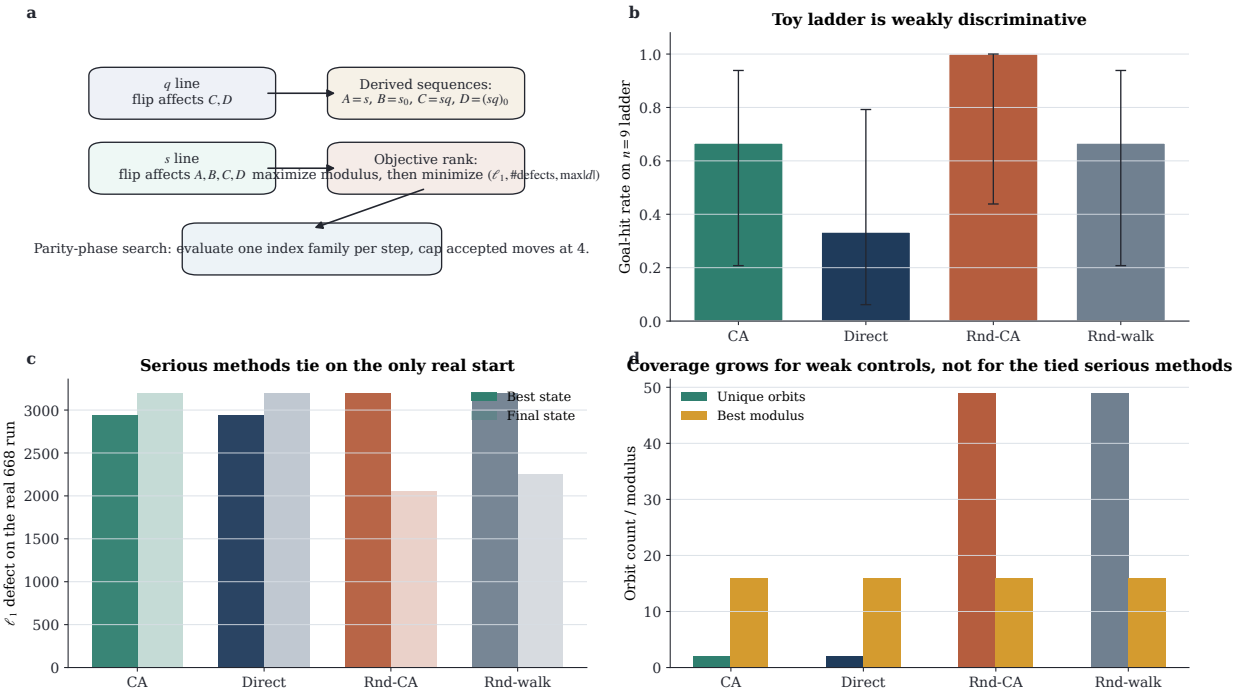

Figure 4: Structured mod-64 precursor results. The toy $n = 9$ ladder is too weak to support a mechanism claim because random controls also solve most starts. The decisive packet is the single real degraded order-668 seed, where `parallel_gain_ca` and `direct_greedy` tie exactly on every load-bearing metric. This closes the flat seed-repair channel before the literal cellar reopen begins.

non-CA baseline. In the literal cellar branch the negative evidence is structural. The boundary-debt state already captures the exact completion information of the implemented encoding, so the dyadic stack has no exact work left to do. On the solved control family that structural claim is visible as frontier exactness and prefix-identifying static state; on the real-anchor packet it is visible as exhaustive zero frontier.

This kind of negative result is useful because it prevents the phrase "cellar automata" from acquiring evidentiary credit it has not earned. The executed branch really did change the representation and really did implement a pushdown-style memory discipline. It just did not buy a new exact information channel on the panels that matter here. That is a stronger conclusion than merely failing to solve the problem with a few heuristic runs.

8.2 Limitations

Several limitations must remain explicit.

1. The sufficiency theorem applies to the implemented boundary-debt state, which is intentionally strong. It does not prove that every weaker non-stack summary would be equivalent.

2. The real-anchor stress packet contains zero exact completions on all eight panels. That is meaningful as a no-go for this encoding, but it does not create a positive control for transfer or proof reuse.

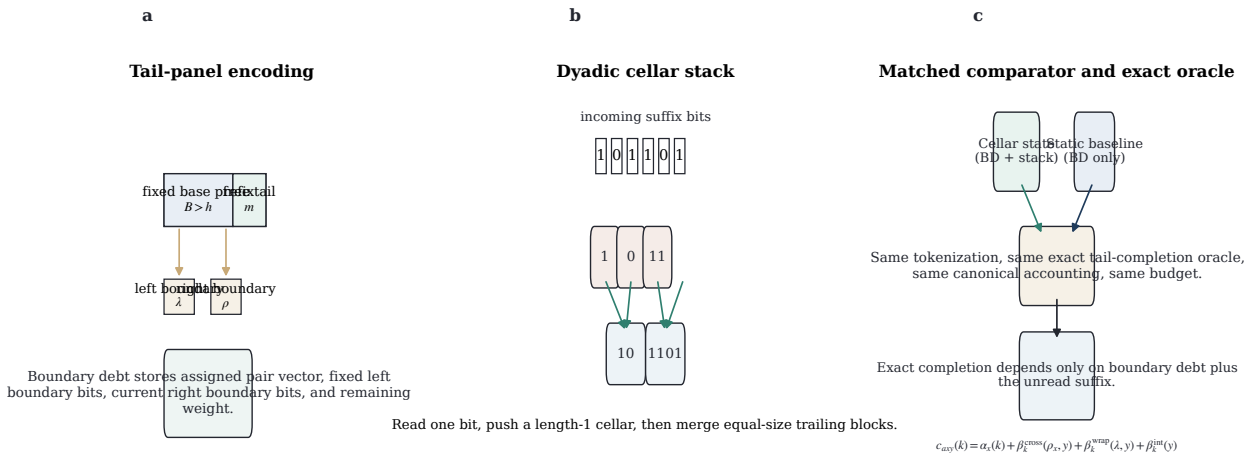

Figure 5: Executed literal cellar encoding. (a) A tail panel fixes a canonical prefix of length $B > h$ and leaves only a contiguous free tail. The matched static residual state stores the zero-padded assigned pair vector together with fixed left boundary bits, current right boundary bits, and remaining weight. (b) The cellar memory reads one suffix bit at a time, pushes a length-1 block, and merges equal-size trailing blocks into dyadic summaries. (c) Both the stack view and the static view feed the same exact completion oracle; the decomposition formula proved in Lemma 5.2 shows why the boundary terms and suffix-internal terms are the only unresolved contributions.

3. The precursor H1 control packet still has exact-hit rate 0/16 for both serious methods under the locked budget. It ranks heuristics but does not demonstrate solver competence under that budget.

4. The structured H2 packet remains underpowered on the real order-668 task because it contains only one degraded start. That is enough to reject the executed repair rule, not enough to classify all structured repair variants.

5. The paper does not benchmark weaker static residual states or alternative symbolic frontiers. Those would be new branches and should not inherit credit from the current no-go.

8.3 What remains open

The most important open question is not "should we spend more compute on this stack?" The answer to that is no. The open question is whether a future branch could change the value proposition away from raw stack memory and toward one of three narrower goals.

First, a weaker static summary might fail where the current boundary-debt state succeeds. That is mathematically possible because Theorem 5.3 is about one exact summary, not all summaries. Second, a future branch might target proof reuse rather than single-panel exactness: the present paper shows that extra feedback cannot help when the static frontier is already exact, but it does not exclude transfer across panels or orders. Third, a different tokenization could make pushdown structure meaningful. None of those possibilities is validated here, and each would require a new matched non-automaton comparator and a new solved same-template control packet.

Table 4: Real-anchor stress packet for the literal cellar branch. Labels are abbreviated as follows: RW30001, RW30004, RW30012, and RW30011 are the four best recorded exact-167/80 target panels from the precursor sweep; SF0 and SF41 are single-flip seed projections; C3 and C5 are three- and five-flip seed projections.

Panel	Anchor kind	Feasible prefixes	Boundary groups	Exact completions	Boundary frontier	Cellar frontier
RW30001	H1 best	1000	1000	0	0	0
RW30004	H1 best	2001	2001	0	0	0
RW30012	H1 best	3002	3002	0	0	0
RW30011	H1 best	3431	3431	0	0	0
SF0	seed projection	3431	3431	0	0	0
SF41	seed projection	3431	3431	0	0	0
C3	seed projection	2001	2001	0	0	0
C5	seed projection	2001	2001	0	0	0

8.4 Societal and scientific implications

The direct societal stakes are low because the paper concerns negative evidence in pure-math search. The scientific-process stakes are less trivial. AI-assisted discovery pipelines can generate novel labels much faster than they generate novel mathematical information. The repo's value here lies in its restraint: it tested a literal cellar branch, proved exactly what its residual state remembers, and stopped once the evidence showed that the stack added no surviving advantage. Cleanly benchmarked negative results of that kind can save downstream compute and reduce the chance that expressive terminology is mistaken for a new algorithmic channel.

9 Conclusion

The repository does not solve the Hadamard order-668 problem. Its earlier flat cellular-automata branches fail or tie under matched controls, and its later literal cellar reopen also fails, but for a more exact reason. On the implemented tail-panel encoding, the matched static boundary-debt summary already determines exact completion sets. The solved control family confirms that this static frontier is exact and prefix-identifying, and the real-anchor stress family shows no exact completion and no stack-only frontier on any recorded panel.

The resulting contribution is narrow but solid. It is not "cellar automata solve Hadamard 668." It is "the executed literal cellar encoding collapses to a matched static residual state and should be retired." Any future reopen should therefore start by changing the representation or the value proposition, not by extending the current stack discipline or spending more budget on the same exact tail-panel oracle.

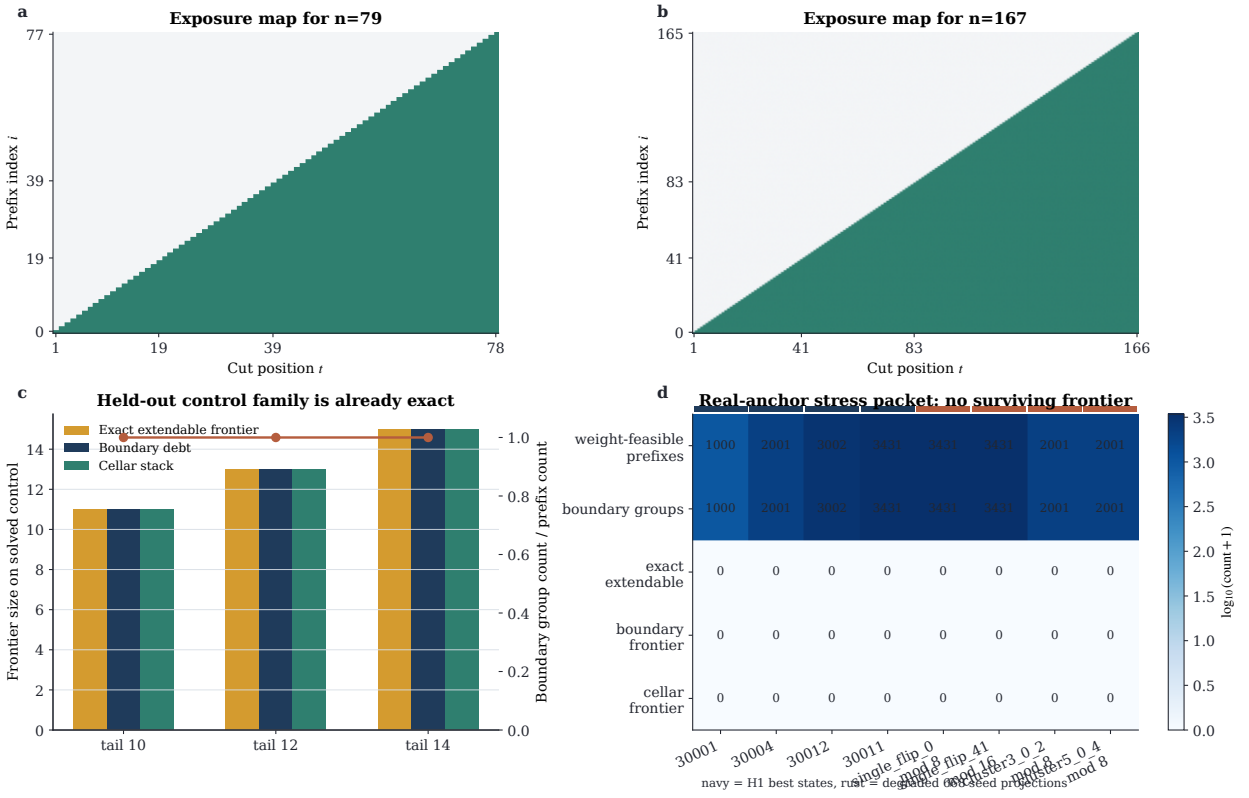

Figure 6: Quantitative results for the literal cellar branch. (a)–(b) Exposure maps for lengths 79 and 167: every admissible prefix coordinate lies inside some left or right boundary window, matching Theorem 5.1. (c) On the solved control family, the exact extendable frontier and the matched static frontier coincide for free-tail lengths 10, 12, and 14; the overlaid line at ratio 1 shows that the static boundary-debt signature is already prefix-identifying. (d) On the eight real-anchor panels, thousands of feasible prefixes are scanned, but the exact-completion, boundary-frontier, and cellar-frontier rows remain identically zero. The top two rows also show that the static signature is injective on the recorded panels, so the no-go cannot be rescued by claiming hidden stack-only classes.

A Additional Proof Detail

A.1 Why the right boundary word is sufficient for crossing terms

In Lemma 5.2, the crossing term at shift k uses the assigned indices $t-k, t-k+1, \ldots, t-1$. The right boundary word ρ_x has length h, and $k \le h$, so those k indices are exactly the last k entries of ρ_x. This is why the full assigned prefix does not need to be stored explicitly once α_x and ρ_x are known.

A.2 Why the wrap term only touches the fixed left boundary

Because the panel fixes more than half of the word, $B > h$, every wrap target index $j \in \{0, \ldots, k-1\}$ lies inside the fixed base prefix, not inside the unread suffix. Hence the wrap contribution depends

on the fixed left boundary word λ and the tail bits near the end of the unread suffix, but not on any deeper stack summary.

B Extended Experimental Tables

B.1 Prefix-identifying behavior on the recorded panels

For every executed panel reported in `results/experiments/cellar_phase6.json` and recomputed in `results/analysis/cellar_paper_metrics.json`, the number of boundary-debt groups equals the number of weight-feasible prefixes. This is stronger than the main theorem requires. The theorem needs only exact completion-set sufficiency; the data show injectivity of the static signature on the recorded panels.

B.2 Writer-pass recomputations

During this revision we reran the literal cellar analysis for the solved control family of free-tail lengths 10, 12, and 14, and regenerated the cellar figures programmatically with

```
python3 -m hadamard668.cellar \
  --output results/experiments/cellar_phase6.json \
  --target-limit 4
python3 figures/generate_cellar_figures.py
```

which also writes `results/analysis/cellar_paper_metrics.json`. The writer pass did not rerun the full H1/H2 experiments because their decisive summaries already exist as checked-in JSON artifacts and are unchanged by the cellar rewrite.

C Hyperparameter and Design-Scope Debt

The literal cellar branch has no tunable move schedule analogous to the precursor CA rules, but it still has representation debt.

1. The implemented state stores an especially strong boundary summary. That was the right conservative choice for a first falsification test, but it likely accelerates collapse to static exactness.
2. The executed panels all use contiguous free tails of lengths 10, 12, or 14. Another tokenization could behave differently.
3. The exact oracle is exhaustive within each panel. A future transfer-oriented branch might seek to learn or approximate reusable constraints instead of recomputing exact completion from scratch.

These are not excuses for the present branch. They are simply the boundaries of the conclusion proved here.

References

Rajeev Alur and P. Madhusudan. Visibly pushdown languages. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing*, pages 202–211, 2004. doi: 10.1145/1007352.1007390. URL https://doi.org/10.1145/1007352.1007390.

Curtis Bright, Ilias S. Kotsireas, and Vijay Ganesh. A SAT+CAS method for enumerating williamson matrices of even order. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.12203. URL https://doi.org/10.1609/aaai.v32i1.12203.

Gregory M. Constantine and Rodica R. Constantine. Convolution numbers: The cyclic case. *arXiv preprint arXiv:2501.18066*, 2025. URL https://arxiv.org/abs/2501.18066.

Dragomir Z. Djoković and Ilias S. Kotsireas. Goethals–seidel difference families with symmetric or skew base blocks. *Mathematics in Computer Science*, 12(4):373–388, 2018. doi: 10.1007/s11786-018-0381-1. URL https://doi.org/10.1007/s11786-018-0381-1.

Shalom Eliahou. A 64-modular hadamard matrix of order 668. *Australasian Journal of Combinatorics*, 93(2):422–427, 2025. URL https://ajc.maths.uq.edu.au/pdf/93/ajc_v93_p422.pdf.

Shalom Eliahou and Michel Kervaire. A survey on modular hadamard matrices. *Discrete Mathematics*, 302(1–3):85–106, 2005. doi: 10.1016/j.disc.2004.08.013. URL https://doi.org/10.1016/j.disc.2004.08.013.

Patrick Fitzpatrick and Henry O'Keeffe. Williamson type hadamard matrices with circulant components. *Discrete Mathematics*, 346(12):113615, 2023. doi: 10.1016/j.disc.2023.113615. URL https://doi.org/10.1016/j.disc.2023.113615.

Maxime Gadouleau, Luca Mariot, and Stjepan Picek. Bent functions from cellular automata. *IACR Cryptology ePrint Archive*, 2020. URL https://eprint.iacr.org/2020/1272.

Kathy J. Horadam. *Hadamard Matrices and Their Applications*. Princeton University Press, 2007.

Luca Manzoni, Luca Mariot, and Giuliamaria Menara. Combinatorial designs and cellular automata: A survey. *arXiv preprint arXiv:2503.10320*, 2025. doi: 10.48550/arXiv.2503.10320. URL https://arxiv.org/abs/2503.10320.

Luca Mariot, Maximilien Gadouleau, Enrico Formenti, and Alberto Leporati. Mutually orthogonal latin squares based on cellular automata. *Designs, Codes and Cryptography*, 88(2):391–411, 2020. doi: 10.1007/s10623-019-00689-8. URL https://doi.org/10.1007/s10623-019-00689-8.

Luca Mariot, Martina Saletta, Alberto Leporati, and Luca Manzoni. Heuristic search of (semi-)bent functions based on cellular automata. *Natural Computing*, 2022. doi: 10.1007/s11047-022-09885-3. URL https://doi.org/10.1007/s11047-022-09885-3.

Andriyan Bayu Suksmono. Finding a hadamard matrix by simulated annealing of spin-vectors. *arXiv preprint arXiv:1606.03815*, 2016. URL https://arxiv.org/abs/1606.03815.

Andriyan Bayu Suksmono. Finding a hadamard matrix by simulated quantum annealing. *Entropy*, 20(2):141, 2018. doi: 10.3390/e20020141. URL https://doi.org/10.3390/e20020141.

Andriyan Bayu Suksmono. A quantum approximate optimization method for finding hadamard matrices. *Scientific Reports*, 15:18778, 2025. doi: 10.1038/s41598-025-18778-1. URL https://doi.org/10.1038/s41598-025-18778-1.

Andriyan Bayu Suksmono and Yuichiro Minato. Finding hadamard matrices by a quantum annealing machine. *Scientific Reports*, 9:17387, 2019. doi: 10.1038/s41598-019-50473-w. URL https://doi.org/10.1038/s41598-019-50473-w.

Andriyan Bayu Suksmono and Yuichiro Minato. Quantum computing formulation of some classical hadamard matrix searching methods and its implementation on a quantum computer. *Scientific Reports*, 12:749, 2022. doi: 10.1038/s41598-021-03586-0. URL https://doi.org/10.1038/s41598-021-03586-0.